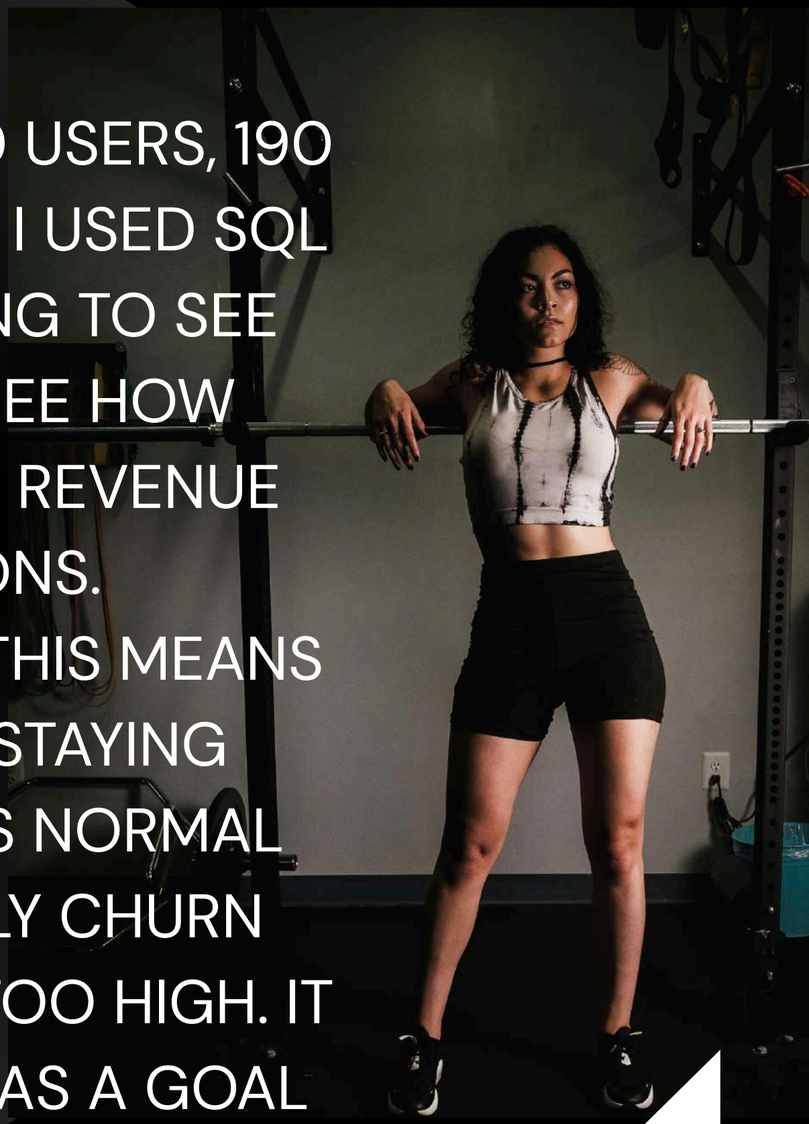
SUBSCRIPTION CHURN REVENUE & COHORT ANALYSIS

(GYM AND FITNESS DATASET)

INTRODUCTION

I LOOKED THROUGH A SUBSCRIPTION DATASET THAT HAS 120 USERS, 190 SUBSCRIPTION ENTRIES AND 2,180 PAYMENT TRANSACTIONS. I USED SQL TOOLS LIKE JOINS, AGGREGATIONS AND COHORT GROUPING TO SEE WHAT WAS HAPPENING. I WANTED TO MEASURE CHURN SEE HOW DIFFERENT ACQUISITION CHANNELS KEPT PEOPLE LOOK AT REVENUE TRENDS AND FIND THE BEST CUSTOMERS AND REGIONS.

THE MAIN THING I FOUND IS THAT THE CHURN RATE IS 60%. THIS MEANS MOST SUBSCRIPTIONS IN THIS DATASET HAVE ENDED OF STAYING ACTIVE. A 60% CHURN RATE IS MUCH HIGHER THAN WHAT'S NORMAL FOR SAAS COMPANIES. SAAS COMPANIES HAVE A MONTHLY CHURN RATE, BETWEEN 5–7%. THIS MEANS THE CHURN RATE IS FAR TOO HIGH. IT IS CLEAR THAT THE TEAM NEEDS TO FOCUS ON RETENTION AS A GOAL INSTEAD OF JUST FINDING NEW USERS.

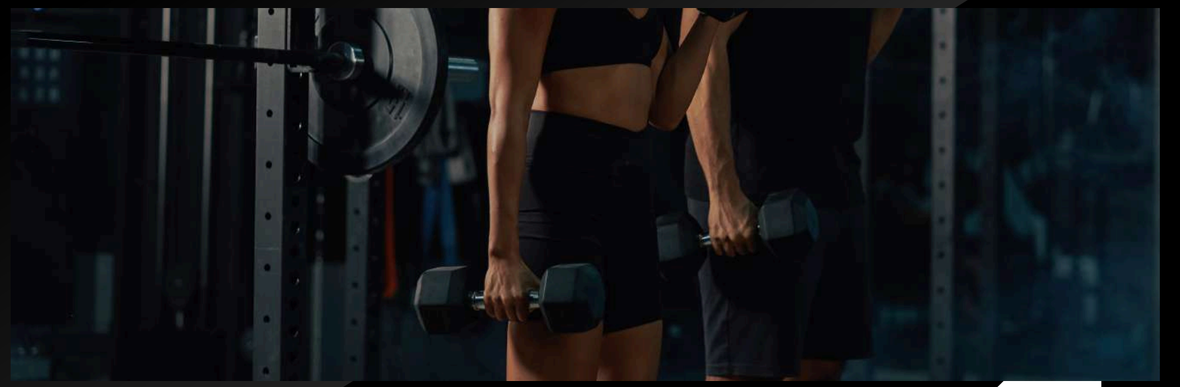


HOW MANY TOTAL USERS DO WE HAVE ?

```
mysql> SELECT COUNT(*) AS total_members FROM users;
+---------------+
| total_members |
+---------------+
|           120 |
+---------------+
1 row in set (0.02 sec)
```

HOW MANY MEMBERSHIPS ARE CURRENTLY ACTIVE VS. CANCELLED?

```
mysql> select churn , count(*) as membership_count from subscriptions group by churn;
+-------+------------------+
| churn | membership_count |
+-------+------------------+
| NO    |               76 |
| YES   |              114 |
+-------+------------------+
2 rows in set (0.00 sec)
```



WHAT IS THE TOTAL REVENUE COLLECTED PER PAYMENT METHOD?

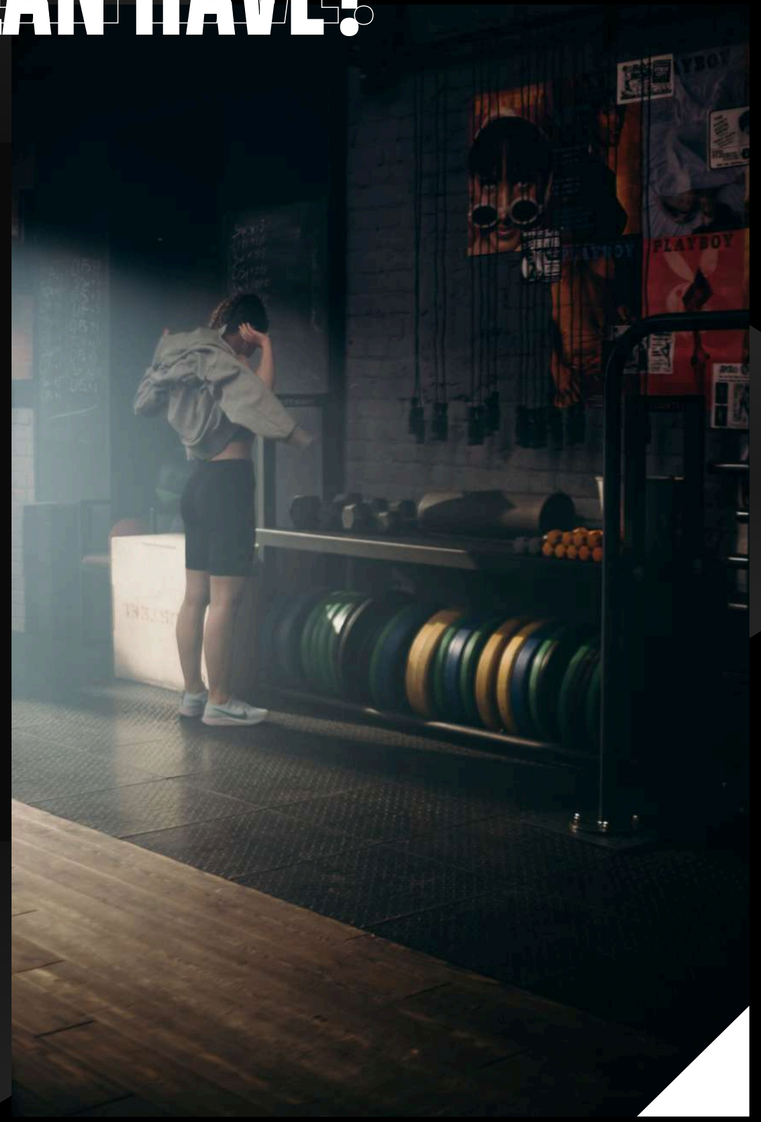
```
mysql> select payment_method , SUM(amount) as total_revenue from payments
    -> GROUP BY payment_method
    -> ORDER BY TOTAL_REVENUE DESC;
+----------------+--------------------+
| payment_method | total_revenue      |
+----------------+--------------------+
| apple_pay      |           20297.56 |
| debit_card     |  19847.19999999999 |
| paypal         |  19782.92000000001 |
| bank_transfer  | 19328.889999999996 |
| credit_card    | 18586.380000000005 |
+----------------+--------------------+
5 rows in set (0.01 sec)
```

WHICH COUNTRY HAS THE MOST USERS?

```
mysql> SELECT country, COUNT(*) AS user_count
    -> FROM users
    -> GROUP BY country
    -> ORDER BY user_count DESC
    -> LIMIT 1;
+---------+------------+
| country | user_count |
+---------+------------+
| UK      |         13 |
+---------+------------+
1 row in set (0.05 sec)
```

HOW MANY SUBSCRIPTIONS DOES EACH PLAN HAVE?

```
mysql> SELECT plan_name, COUNT(*) AS subscription_count
    -> FROM subscriptions
    -> GROUP BY plan_name
    -> ORDER BY subscription_count DESC;
+-----------+--------------------+
| plan_name | subscription_count |
+-----------+--------------------+
| Pro       |                 44 |
| Enterprise |                41 |
| Basic     |                 37 |
| Standard  |                 37 |
| Premium   |                 31 |
+-----------+--------------------+
5 rows in set (0.02 sec)
```



LIST THE 10 MOST RECENT CHURNED SUBSCRIPTIONS.

```
mysql> SELECT s.subscription_id, s.user_id, s.plan_name, s.end_date
    -> FROM subscriptions s
    -> WHERE s.is_current = 'FALSE'
    -> ORDER BY s.end_date DESC
    -> LIMIT 10;
+-----------------+---------+-----------+------------+
| subscription_id | user_id | plan_name | end_date   |
+-----------------+---------+-----------+------------+
|             103 |      64 | Pro       | 2026-08-21 |
|             102 |      63 | Basic     | 2026-08-21 |
|             185 |     118 | Pro       | 2026-08-21 |
|             145 |      93 | Premium   | 2026-08-21 |
|             166 |     107 | Pro       | 2026-08-21 |
|              39 |      21 | Enterprise| 2026-08-21 |
|             135 |      87 | Standard  | 2026-08-15 |
|              38 |      21 | Premium   | 2026-08-10 |
|               6 |       4 | Enterprise| 2026-07-20 |
|              94 |      59 | Enterprise| 2026-07-13 |
+-----------------+---------+-----------+------------+
10 rows in set (0.01 sec)
```

HOW MANY MEMBERS ARE SUBSCRIBED TO EACH MEMBERSHIP TYPE?

```
mysql> SELECT plan_name , COUNT(*) AS total_members
    -> FROM subscriptions
    -> GROUP BY plan_name;
+-------------+---------------+
| plan_name   | total_members |
+-------------+---------------+
| Enterprise |            41 |
| Basic      |            37 |
| Standard   |            37 |
| Premium    |            31 |
| Pro        |            44 |
+-------------+---------------+
5 rows in set (0.04 sec)
```

WHAT PERCENT OF ALL SUBSCRIPTIONS HAVE CHURNED?

```
mysql> SELECT
    ->     SUM(CASE WHEN is_current = 'FALSE' THEN 1 ELSE 0 END) AS is_current_count,
    ->     COUNT(*) AS total_count,
    ->     ROUND(100.0 * SUM(CASE WHEN is_current = 'YES' THEN 1 ELSE 0 END) / COUNT(*), 2)
    -> AS is_current_rate_pct
    -> FROM subscriptions;
+------------------+-------------+---------------------+
| is_current_count | total_count | is_current_rate_pct |
+------------------+-------------+---------------------+
|              114 |         190 |                0.00 |
+------------------+-------------+---------------------+
1 row in set (0.00 sec)
```

FOR EACH ACQUISITION CHANNEL, HOW MANY SUBSCRIPTIONS CHURNED VS. STAYED ACTIVE?

```
mysql> SELECT
    ->     u.acquisition_channel,
    ->     s.is_current,
    ->     COUNT(*) AS subscription_count
    -> FROM subscriptions s
    -> JOIN users u ON u.user_id = s.user_id
    -> GROUP BY u.acquisition_channel, s.is_current
    -> ORDER BY u.acquisition_channel;
+--------------------+------------+--------------------+
| acquisition_channel | is_current | subscription_count |
+--------------------+------------+--------------------+
| affiliate          | FALSE      |                 16 |
| affiliate          | TRUE       |                 12 |
| direct             | FALSE      |                 18 |
| direct             | TRUE       |                  9 |
| email_campaign     | FALSE      |                 10 |
| email_campaign     | TRUE       |                 13 |
| organic_search     | FALSE      |                 17 |
| organic_search     | TRUE       |                 10 |
| paid_search        | FALSE      |                 23 |
| paid_search        | TRUE       |                 13 |
| referral           | FALSE      |                 14 |
| referral           | TRUE       |                  9 |
| social_media       | FALSE      |                 16 |
| social_media       | TRUE       |                 10 |
+--------------------+------------+--------------------+
14 rows in set (0.00 sec)
```

. WHAT IS THE AVERAGE NUMBER OF DAYS A SUBSCRIPTION LASTED BEFORE IT CHURNED?

```
mysql> SELECT
    -> ROUND(AVG(DATEDIFF(end_date, start_date)), 1) AS avg_days_before_churn
    -> FROM subscriptions
    -> WHERE churn = 'Yes';
+-----------------------+
| avg_days_before_churn |
+-----------------------+
|                 113.2 |
+-----------------------+
1 row in set (0.00 sec)
```

. WHAT IS THE AVERAGE NUMBER OF DAYS A SUBSCRIPTION LASTED BEFORE IT CHURNED, BROKEN DOWN BY PLAN?

```
mysql> SELECT
    ->     plan_name,
    ->     ROUND(AVG(DATEDIFF(end_date, start_date)), 1) AS avg_days_before_is_current
    -> FROM subscriptions
    -> WHERE is_current = 'FALSE'
    -> GROUP BY plan_name
    -> ORDER BY avg_days_before_is_current DESC;
+------------+-----------------------------+
| plan_name  | avg_days_before_is_current  |
+------------+-----------------------------+
| Standard   |                       141.8 |
| Basic      |                       111.7 |
| Enterprise |                       107.8 |
| Pro        |                        99.6 |
| Premium    |                        98.5 |
+------------+-----------------------------+
5 rows in set (0.01 sec)
```

HOW MUCH HAS EACH USER PAID IN TOTAL? LABEL ANYONE UNDER $50 TOTAL AS "LOW VALUE."

```
mysql> SELECT
    -> u.user_id,
    -> SUM(p.amount) as total_paid,
    -> CASE
    ->   WHEN SUM(p.amount) < 50 THEN 'Low value'
    ->   ELSE 'Standard'
    -> END AS value_flag
    -> FROM users u
    -> JOIN payments p ON p.user_id = u.user_id
    -> GROUP BY u.user_id
    -> ORDER BY total_paid ASC;
+---------+--------------------+------------+
| user_id | total_paid         | value_flag |
+---------+--------------------+------------+
|      99 |              10.26 | Low value  |
|      80 |              20.22 | Low value  |
|      63 |              20.39 | Low value  |
|      31 |              20.91 | Low value  |
|      36 |              39.13 | Low value  |
|      61 |              39.84 | Low value  |
|      90 |              40.17 | Low value  |
|     106 |              40.98 | Low value  |
|      64 |              49.77 | Low value  |
|      56 |  59.93000000000001 | Standard   |
|      77 |              60.61 | Standard   |
|      55 |               70.2 | Standard   |
|      18 |  79.17999999999999 | Standard   |
|     114 |              79.83 | Standard   |
|       1 |              97.92 | Standard   |
|      54 |              98.19 | Standard   |
|      52 |  98.28999999999999 | Standard   |
|     116 |              99.12 | Standard   |
```

WHAT IS TOTAL REVENUE COLLECTED PER MONTH?

```
mysql> SELECT
    ->     DATE_FORMAT(payment_date, '%Y-%m') AS revenue_month,
    ->     SUM(amount) AS total_revenue
    -> FROM payments
    -> GROUP BY revenue_month
    -> ORDER BY revenue_month
    -> LIMIT 10;
+---------------+--------------------+
| revenue_month | total_revenue      |
+---------------+--------------------+
| 2023-01       |              38.56 |
| 2023-02       |             241.42 |
| 2023-03       |             284.89 |
| 2023-04       |              269.1 |
| 2023-05       | 487.16999999999996 |
| 2023-06       |  642.8199999999999 |
| 2023-07       |  936.4499999999999 |
| 2023-08       |            1217.21 |
| 2023-09       | 1155.4699999999998 |
| 2023-10       | 1305.9800000000002 |
+---------------+--------------------+
10 rows in set (0.00 sec)
```

LIST EVERY CHURNED SUBSCRIPTION ALONG WITH HOW MUCH THAT USER PAID IN TOTAL (JOIN SUBSCRIPTIONS TO PAYMENT TOTALS)

```
mysql> SELECT
    ->     s.user_id,
    ->     s.plan_name,
    ->     s.end_date,
    ->     SUM(p.amount) AS total_paid_by_user
    -> FROM subscriptions s
    -> LEFT JOIN payments p ON p.user_id = s.user_id
    -> WHERE s.churn = 'YES'
    -> GROUP BY s.user_id , s.plan_name , s.end_date
    -> ORDER BY total_paid_by_user ASC;
+---------+------------+------------+--------------------+
| user_id | plan_name  | end_date   | total_paid_by_user |
+---------+------------+------------+--------------------+
|      80 | Basic      | 2025-04-27 |              20.22 |
|      63 | Basic      | 2026-08-21 |              20.39 |
|      36 | Standard   | 2024-11-25 |              39.13 |
|      61 | Standard   | 2024-07-23 |              39.84 |
|      90 | Standard   | 2024-06-01 |              40.17 |
|     106 | Basic      | 2026-05-02 |              40.98 |
|      64 | Pro        | 2026-08-21 |              49.77 |
|      56 | Premium    | 2023-09-22 |  59.93000000000001 |
|      77 | Basic      | 2026-06-26 |  60.61000000000001 |
|      18 | Standard   | 2023-04-29 |  79.17999999999999 |
|      18 | Basic      | 2023-07-29 |  79.17999999999999 |
|     114 | Standard   | 2025-10-18 |              79.83 |
|      54 | Pro        | 2025-09-15 |              98.19 |
|      52 | Pro        | 2025-06-13 |  98.28999999999999 |
|     116 | Pro        | 2025-10-23 |              99.12 |
|     119 | Premium    | 2023-06-04 |              99.38 |
|     119 | Basic      | 2023-09-07 |              99.38 |
|     100 | Pro        | 2025-05-04 |  99.77000000000001 |
|      76 | Pro        | 2024-07-26 |             100.38 |
```

RANK EVERY USER BY THEIR TOTAL LIFETIME REVENUE (HIGHEST SPENDER = RANK 1).

```
mysql> SELECT
    ->     u.user_id,
    ->     SUM(p.amount) AS lifetime_revenue,
    ->     RANK() OVER (ORDER BY SUM(p.amount) DESC) AS revenue_rank
    -> FROM users u
    -> JOIN payments p ON p.user_id = u.user_id
    -> GROUP BY u.user_id
    -> ORDER BY revenue_rank
    -> LIMIT 10;
+---------+--------------------+--------------+
| user_id | lifetime_revenue   | revenue_rank |
+---------+--------------------+--------------+
|      84 |            4267.67 |            1 |
|      73 |               3987 |            2 |
|      72 |  3858.669999999999 |            3 |
|       3 | 3494.8099999999995 |            4 |
|      69 | 3344.2799999999997 |            5 |
|      14 |            3097.54 |            6 |
|      79 | 3094.6499999999996 |            7 |
|      85 |            2882.85 |            8 |
|      15 |            2452.41 |            9 |
|      38 |            2149.09 |           10 |
+---------+--------------------+--------------+
10 rows in set (0.00 sec)
```

SAME IDEA, BUT RANKED SEPARATELY WITHIN EACH COUNTRY (TOP SPENDER PER COUNTRY GETS RANK 1 IN THAT COUNTRY).

```
mysql> SELECT
    ->     u.user_id,
    ->     u.country,
    ->     SUM(p.amount) AS lifetime_revenue,
    ->     RANK() OVER (PARTITION BY u.country ORDER BY SUM(p.amount) DESC) AS country_rank
    -> FROM users u
    -> JOIN payments p ON p.user_id = u.user_id
    -> GROUP BY u.user_id, u.country
    -> ORDER BY u.country, country_rank;
+---------+-------------+--------------------+--------------+
| user_id | country     | lifetime_revenue   | country_rank |
+---------+-------------+--------------------+--------------+
|     103 | Australia   | 1117.0100000000002 |            1 |
|      53 | Australia   |             941.28 |            2 |
|      42 | Australia   | 219.10999999999999 |            3 |
|       9 | Australia   |              204.1 |            4 |
|      26 | Australia   |             128.97 |            5 |
|      46 | Australia   |  99.61000000000001 |            6 |
|      77 | Australia   |              60.61 |            7 |
|     104 | Brazil      |            1306.44 |            1 |
|     105 | Brazil      |             1072.8 |            2 |
|      97 | Brazil      |  853.2299999999999 |            3 |
|      68 | Brazil      |  686.2199999999999 |            4 |
|     110 | Brazil      |              550.4 |            5 |
|      93 | Brazil      |             220.82 |            6 |
|      81 | Brazil      | 190.20999999999995 |            7 |
|      65 | Canada      | 1467.2500000000002 |            1 |
|      59 | Canada      | 1369.9900000000002 |            2 |
```

FOR EACH USER, SHOW EVERY PAYMENT PLUS A RUNNING (CUMULATIVE) TOTAL OF WHAT THEY'VE PAID SO FAR.

```
mysql> SELECT
    -> user_id,
    -> payment_date,
    -> amount,
    -> SUM(amount) OVER (
    -> PARTITION BY user_id
    -> ORDER BY payment_date
    -> ) AS running_total_paid
    -> FROM payments
    -> ORDER BY user_id, payment_date
    -> LIMIT 10;
+---------+--------------+--------+--------------------+
| user_id | payment_date | amount | running_total_paid |
+---------+--------------+--------+--------------------+
|       1 | 2026-08-02   |  97.92 |              97.92 |
|       2 | 2024-07-17   |   9.64 |               9.64 |
|       2 | 2024-08-16   |  10.06 |             116.39 |
|       2 | 2024-08-16   |  96.69 |             116.39 |
|       2 | 2024-09-15   |  103.7 |             220.09 |
|       2 | 2024-09-20   |  19.99 |             240.08 |
|       2 | 2024-10-20   |  19.45 | 259.53000000000003 |
|       2 | 2024-11-19   |  19.91 | 279.44000000000005 |
|       2 | 2024-12-19   |  20.79 |  300.2300000000001 |
|       2 | 2025-01-18   |  20.35 |  320.5800000000001 |
+---------+--------------+--------+--------------------+
10 rows in set (0.01 sec)
```

1GROUP USERS BY THE MONTH THEY SIGNED UP (THEIR "COHORT"), AND COUNT HOW MANY STILL HAVE AN ACTIVE (NON-CHURNED) SUBSCRIPTION TODAY.

```
mysql> WITH cohorts AS (
    ->     SELECT user_id, DATE_FORMAT(signup_date, '%Y-%m') AS cohort_month
    ->
    ->     FROM users
    -> )
    -> SELECT
    ->     c.cohort_month,
    ->     COUNT(*) AS cohort_size,
    ->     SUM(CASE WHEN s.is_current = 'TRUE' THEN 1 ELSE 0 END) AS still_active
    -> FROM cohorts c
    -> JOIN subscriptions s ON s.user_id = c.user_id
    -> GROUP BY c.cohort_month
    -> ORDER BY c.cohort_month
    -> LIMIT 20;
+--------------+-------------+--------------+
| cohort_month | cohort_size | still_active |
+--------------+-------------+--------------+
| 2023-01      |           6 |            1 |
| 2023-02      |           1 |            1 |
| 2023-03      |           3 |            1 |
| 2023-04      |           6 |            3 |
| 2023-05      |           6 |            2 |
| 2023-06      |           6 |            3 |
| 2023-07      |          10 |            4 |
| 2023-08      |           7 |            2 |
| 2023-09      |           3 |            1 |
| 2023-10      |           5 |            3 |
| 2023-11      |           5 |            3 |
| 2024-01      |           8 |            2 |
| 2024-02      |           4 |            2 |
| 2024-03      |           3 |            1 |
| 2024-04      |           4 |            4 |
| 2024-05      |           5 |            2 |
| 2024-06      |           8 |            3 |
| 2024-07      |           4 |            2 |
| 2024-08      |           3 |            1 |
| 2024-09      |           1 |            0 |
+--------------+-------------+--------------+
```

FOR EACH COUNTRY, COMPARE EVERY USER'S REVENUE TO THAT COUNTRY'S AVERAGE REVENUE.

```
mysql> SELECT
    ->     u.user_id,
    ->     u.country,
    ->     SUM(p.amount) AS user_revenue,
    ->     ROUND(AVG(SUM(p.amount)) OVER (PARTITION BY u.country), 2) AS country_avg_revenue
    -> FROM users u
    -> JOIN payments p ON p.user_id = u.user_id
    -> GROUP BY u.user_id, u.country
    -> ORDER BY u.country, user_revenue DESC
    -> LIMIT 20;
+---------+-----------+--------------------+----------------------+
| user_id | country   | user_revenue       | country_avg_revenue  |
+---------+-----------+--------------------+----------------------+
|     103 | Australia | 1117.0100000000002 |                395.81 |
|      53 | Australia |             941.28 |                395.81 |
|      42 | Australia | 219.10999999999999 |                395.81 |
|       9 | Australia |              204.1 |                395.81 |
|      26 | Australia |             128.97 |                395.81 |
|      46 | Australia |  99.61000000000001 |                395.81 |
|      77 | Australia |              60.61 |                395.81 |
|     104 | Brazil    |            1306.44 |                697.16 |
|     105 | Brazil    |             1072.8 |                697.16 |
|      97 | Brazil    |  853.2299999999999 |                697.16 |
|      68 | Brazil    |  686.2199999999999 |                697.16 |
|     110 | Brazil    |              550.4 |                697.16 |
|      93 | Brazil    |             220.82 |                697.16 |
|      81 | Brazil    | 190.20999999999995 |                697.16 |
|      65 | Canada    | 1467.2500000000002 |                716.05 |
|      59 | Canada    | 1369.9900000000002 |                716.05 |
|      74 | Canada    |            1085.55 |                716.05 |
|      50 | Canada    |  756.1800000000001 |                716.05 |
|      28 | Canada    |             731.17 |                716.05 |
|      33 | Canada    |  580.0000000000002 |                716.05 |
+---------+-----------+--------------------+----------------------+
20 rows in set (0.01 sec)
```

CONCLUSION

THIS SQL ANALYSIS OF THE SUBSCRIPTION DATASET TELLS A CLEAR STORY: THE REAL PROBLEM IS NOT GETTING NEW USERS BUT KEEPING THEM. WITH A 60% CHURN RATE THE PLATFORM IS LOSING MOST OF THE SUBSCRIPTIONS IT GETS. THE DATA SHOWS THAT THIS IS NOT A GENERAL PROBLEM BUT ONE WITH SPECIFIC CAUSES WE CAN ACTUALLY FIX.

THREE MAIN FINDINGS STAND OUT FROM THIS PROJECT:

- THE QUALITY OF CHANNELS VARIES A LOT. EMAIL CAMPAIGN USERS CHURN AT 43.5% WHILE DIRECT SIGNUPS CHURN AT 66.7%. THIS 23-POINT GAP SHOWS THAT WE SHOULD SPEND MONEY ON CHANNELS THAT BRING IN USERS WHO ACTUALLY WANT TO STAY.
- MID-TIER PLANS KEEP USERS LONGER THAN PREMIUM TIERS. STANDARD-PLAN USERS STAY 42 DAYS LONGER ON AVERAGE THAN PREMIUM USERS. THIS SUGGESTS THAT WE NEED TO LOOK AT HOW WE WELCOME PREMIUM USERS OR IF THEY FEEL THEY ARE GETTING VALUE INSTEAD OF JUST ASSUMING EXPENSIVE PLANS MEAN MORE LOYALTY.
- REVENUE IS NOT SPREAD OUT EVENLY. A SMALL GROUP OF CUSTOMERS AND TWO COUNTRIES (MEXICO AND SPAIN) MAKE UP A HUGE PART OF THE TOTAL REVENUE. WE MUST TREAT PROTECTING THESE ACCOUNTS AS A PRIORITY FOR RETENTION NOT JUST A GOAL FOR GROWTH.

RECOMMENDATION: I SUGGEST WE FOCUS ON A TARGETED RETENTION PLAN OF A BROAD CAMPAIGN THAT TRIES TO FIX EVERYTHING AT ONCE. WE SHOULD DO CHECK-INS FOR HIGH-LTV ACCOUNTS LOOK CLOSELY AT PREMIUM/PRO ONBOARDING AND PUT MORE MONEY INTO EMAIL-DRIVEN ACQUISITION. SINCE CHURN DEPENDS MUCH ON THE ACQUISITION CHANNEL AND THE PLAN TIER A SEGMENTED APPROACH WILL LIKELY REDUCE CHURN MUCH MORE THAN A ONE-SIZE-FITS-ALL FIX.

THIS PROJECT SHOWS A SQL ANALYTICS WORKFLOW. I WENT FROM RELATIONAL DATA THROUGH JOINS, AGGREGATION, CASE-BASED SEGMENTATION AND WINDOW FUNCTIONS TO FIND THESE BUSINESS-READY FINDINGS AND RECOMMENDATIONS, FOR A STAKEHOLDER-FACING PRESENTATION.